

แบบทดสอบท้ายแล็บ

Custom ViewResolver ใน Spring Boot + Thymeleaf

วิชา CP353002 Principles of Software Design

ชื่อ-นามสกุล: นางสาวณัฐนันท์ บุษดี รหัสนักศึกษา: 673380037-1

คำชี้แจง: ส่วนที่ 1 ปรนัย เลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว ส่วนที่ 2 อัตนัย ตอบเป็นข้อความอธิบาย ส่วนที่ 3 ปฏิบัติ
ให้ลงมือแก้ไขได้จริงในโปรเจกต์ของตัวเอง แล้ว capture หน้าจอผลลัพธ์แปะในกล่องที่เตรียมไว้

ส่วนที่ 1 — ปรนัย (Multiple Choice)

1. ในสถาปัตยกรรม MVC ส่วนใดที่ทำหน้าที่แปลง "ชื่อ view" ให้เป็น path ของไฟล์ view จริง? (2

คะแนน)

- ก. Model
- ข. Controller
- ค. ViewResolver
- ง. DispatcherServlet

2. เมื่อ Controller เขียน return "home"; ค่า "home" ที่ถูกส่งกลับคืออะไร? (2 คะแนน)

- ก. Path ของไฟล์ HTML บนเครื่อง server
- ข. ชื่อ view ให้ตรง (logical view name)
- ค. ชื่อ bean ของ ViewResolver
- ง. ชื่อ method ของ Controller

3. component ใดใน Spring MVC เป็นผู้เรียกใช้ ViewResolver โดยอัตโนมัติ หลังจาก Controller คืนค่าชื่อ view?

(2 คะแนน)

- ก. HandlerMapping
- ข. DispatcherServlet
- ค. SpringTemplateEngine
- ง. HomeController

4. ใน ThymeleafConfig.java เมธอด resolver.setPrefix("classpath:/custom-templates/") ทำหน้าที่อะไร? (2

คะแนน)

- ก. กำหนด URL ที่ Controller จะรับ request
- ข. กำหนดโฟลเดอร์ต้นทางที่ ViewResolver จะค้นหาไฟล์ template

ค. กำหนดชื่อ database connection

ง. กำหนด port ที่แอปพลิเคชันรัน

5. ถ้ามีการลงทะเบียน ViewResolver มากกว่าหนึ่งในบริบท (context) เดียวกัน Spring จะเลือกใช้ตัวไหนก่อน? (2 คะแนน)

ก. ตัวที่ประกาศเป็นตัวแรกในไฟล์เสมอ

ข. ตัวที่มีค่า order ต่ำกว่า (ถูกเรียกก่อน)

ค. สุ่มเลือก

ง. ตัวที่ตั้งชื่อ bean เรียงตามตัวอักษร

6. เพราะเหตุใด Controller ในตัวอย่างนี้จึงใช้ @Controller แทน @RestController? (2 คะแนน)

ก. เพราะ @RestController ใช้กับ Thymeleaf ไม่ได้เลย

ข. เพราะต้องการให้ค่าที่ return ถูกตีความเป็นชื่อ view ไม่ใช่ response body โดยตรง

ค. เพราะ @Controller ทำงานเร็วกว่า

ง. ไม่มีความแตกต่างกันเลย

ส่วนที่ 2 — อรรถนัย (Short Answer / Essay)

1. ใครเป็นผู้เรียกใช้ Custom ViewResolver และเรียกเมื่อไร? จงอธิบายเป็นลำดับขั้นตอน ตั้งแต่ Browser ส่ง request จนได้ HTML response กลับ (5 คะแนน)

DispatcherServlet เรียกใช้ Custom ViewResolver หลังจากที่ HomeController คืนค่า "home" ซึ่งเป็น Logical View Name

1. ผู้ใช้เปิดเว็บเบราว์เซอร์และส่ง Request ไปยัง URL เช่น http://localhost:8080/
2. Request ถูกส่งไปยัง DispatcherServlet
3. DispatcherServlet ส่ง Request ไปยัง HomeController
4. HomeController ประมวลผลและเพิ่มข้อมูลลงใน Model
5. HomeController คืนค่า "home" ซึ่งเป็น Logical View Name
6. DispatcherServlet เรียกใช้ Custom ViewResolver เพื่อแปลงชื่อ "home" ให้เป็นไฟล์จริง
7. ViewResolver ใช้ prefix และ suffix รวมกันเป็น classpath:/custom-templates/home.html
8. Thymeleaf ประมวลผลไฟล์ HTML พร้อมข้อมูลใน Model
9. ส่ง HTML กลับไปยัง Browser เพื่อแสดงผล

2. จงอธิบายว่าเหตุใดการแยก ViewResolver ออกจาก Controller จึงสอดคล้องกับหลักการ separation of concerns และ dependency inversion ใน Principles of Software Design (5 คะแนน)

การแยก ViewResolver ออกจาก Controller ทำให้แต่ละส่วนมีหน้าที่ของตนเอง (Separation of Concerns)

Controller มีหน้าที่รับ Request ประมวลผลข้อมูล และคืนชื่อ View เท่านั้น โดยไม่ต้องรู้ว่าไฟล์ HTML อยู่ที่ตำแหน่งใด

ส่วน ViewResolver มีหน้าที่แปลงชื่อ View ไปเป็นไฟล์จริง ทำให้สามารถเปลี่ยนตำแหน่งของ Template หรือเปลี่ยน Template Engine ได้โดยไม่ต้องแก้ไข Controller ซึ่งสอดคล้องกับหลัก Dependency Inversion เพราะ Controller พึ่งพาเพียงชื่อ View เชิงตรรกะ ไม่ได้พึ่งพาดำแหน่งของไฟล์จริง

3. หากต้องการเพิ่ม view ใหม่ชื่อ "about" โดยยังใช้ custom ViewResolver ตัวเดิม ต้องสร้างหรือแก้ไขไฟล์ใดบ้าง? ระบุชื่อไฟล์และ path ให้ครบถ้วน (3 คะแนน)

สร้างไฟล์ about.html ที่ path src/main/resources/custom-templates ใส่โค้ดต่อไปนี้

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
```

```
<meta charset="UTF-8">
<title>About</title>
</head>
<body>

  <h1 th:text="${title}">About</h1>

  <p th:text="${description}">
    Description
  </p>

</body>
</html>
```

และต้องแก้ไข HomeController.java ที่ src/main/java/com/example/demo/controller/HomeController.java โดยเพิ่มโค้ดต่อไปนี้ใน class HomeController

```
@GetMapping("/about")
public String about(Model model){

    model.addAttribute("title","About");

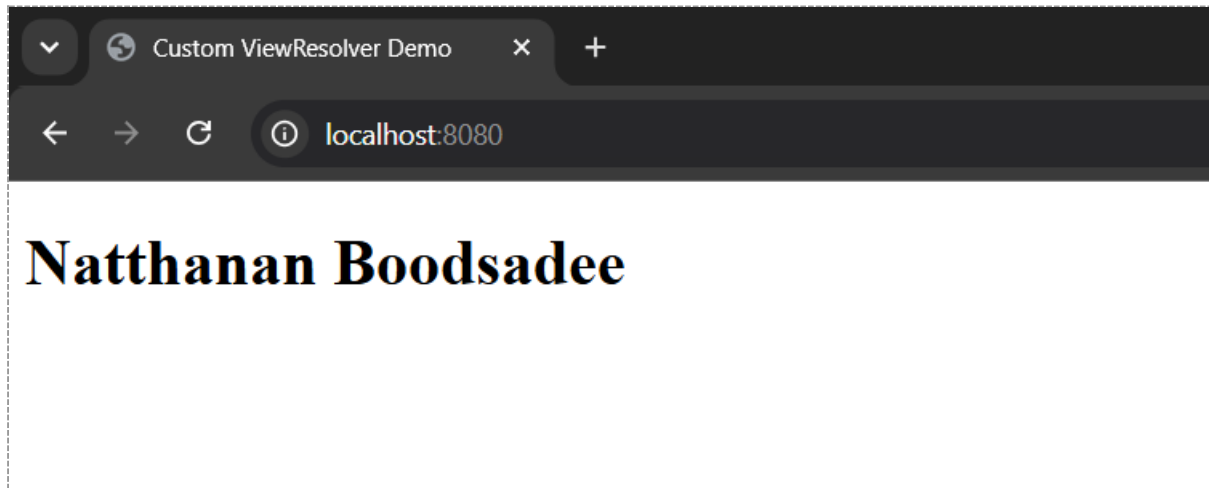
    model.addAttribute(
        "description",
        "This page uses the same Custom ViewResolver."
    );

    return "about";
}
```

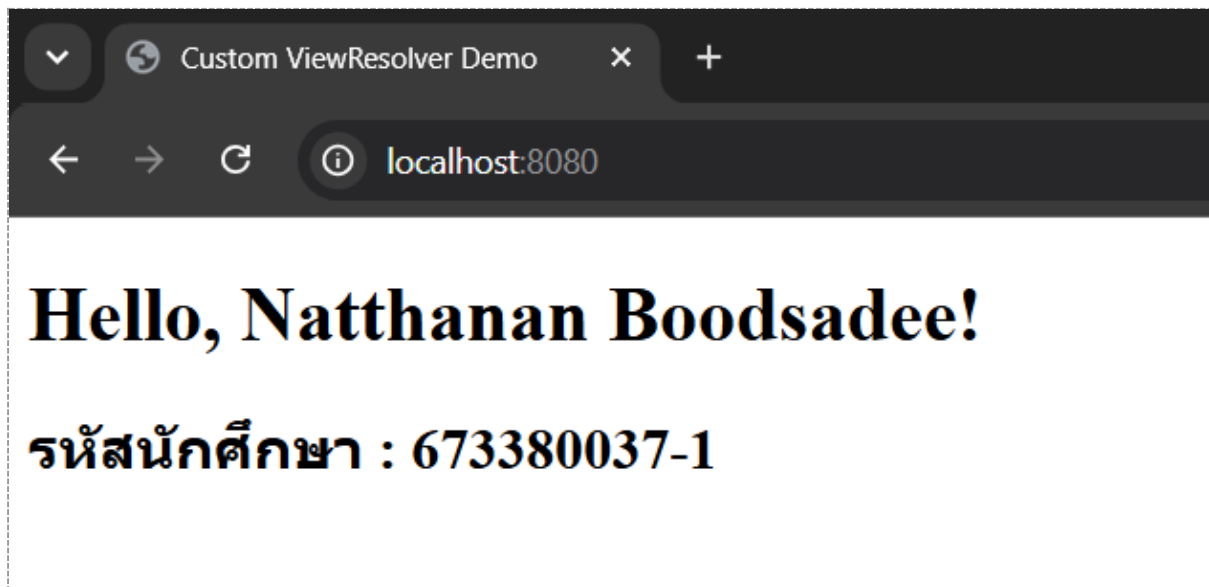
ส่วนที่ 3 — ปฏิบัติ (แก้โค้ดจริง + แบนภาพหน้าจอ)

ทำตามคำสั่งแต่ละข้อในโปรเจกต์ของตัวเอง รันด้วย `mvn spring-boot:run` แล้ว capture หน้าจอผลลัพธ์แปะในกล่องเส้นประด้านล่างแต่ละข้อ

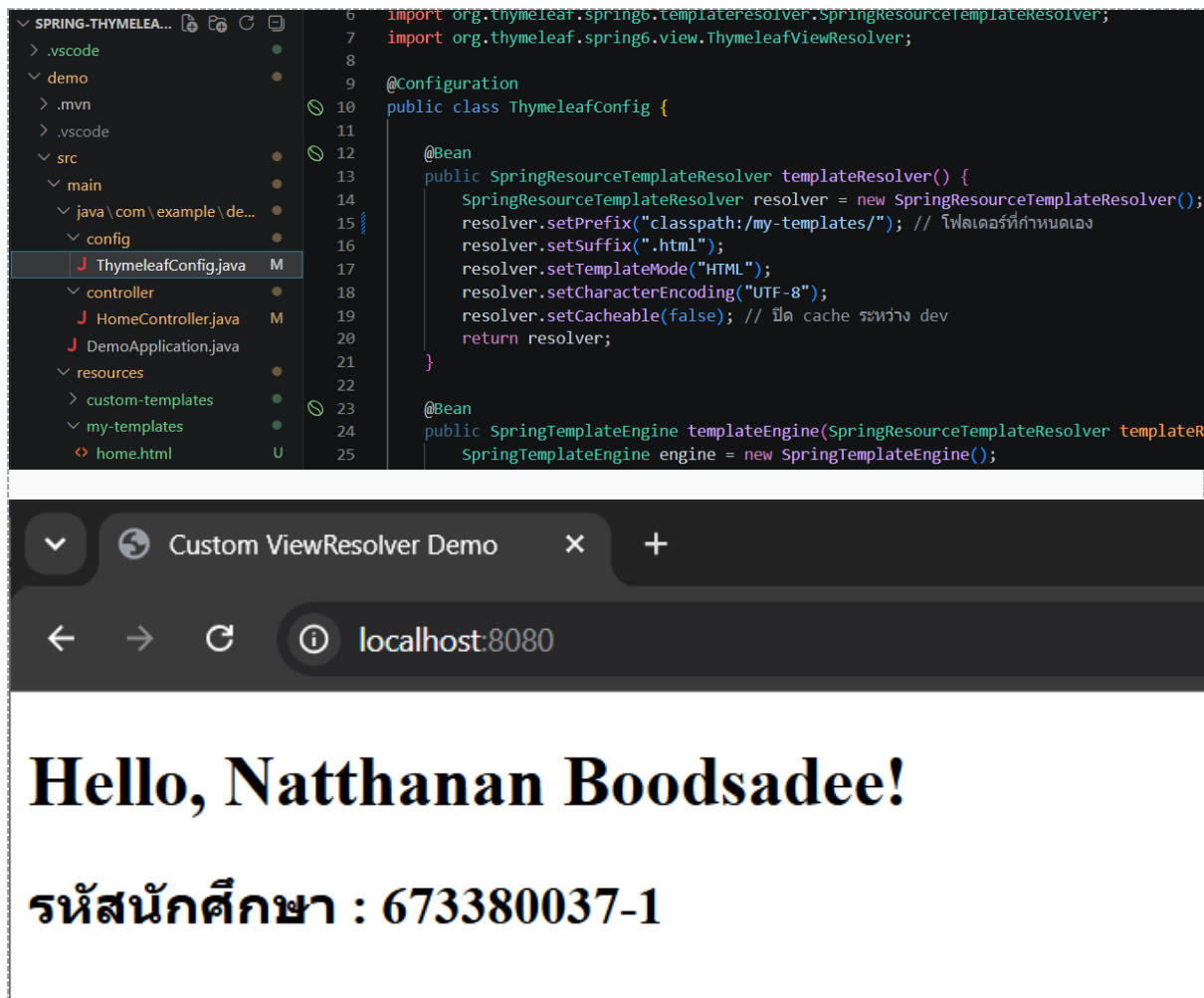
1. แก้ไขค่าตัวแปร `message` ใน `HomeController.java` ให้แสดงชื่อ-นามสกุลของตัวเองแทนข้อความเดิม แล้ว capture หน้าเว็บ `http://localhost:8080/` ที่แสดงชื่อของตัวเอง (3 คะแนน)



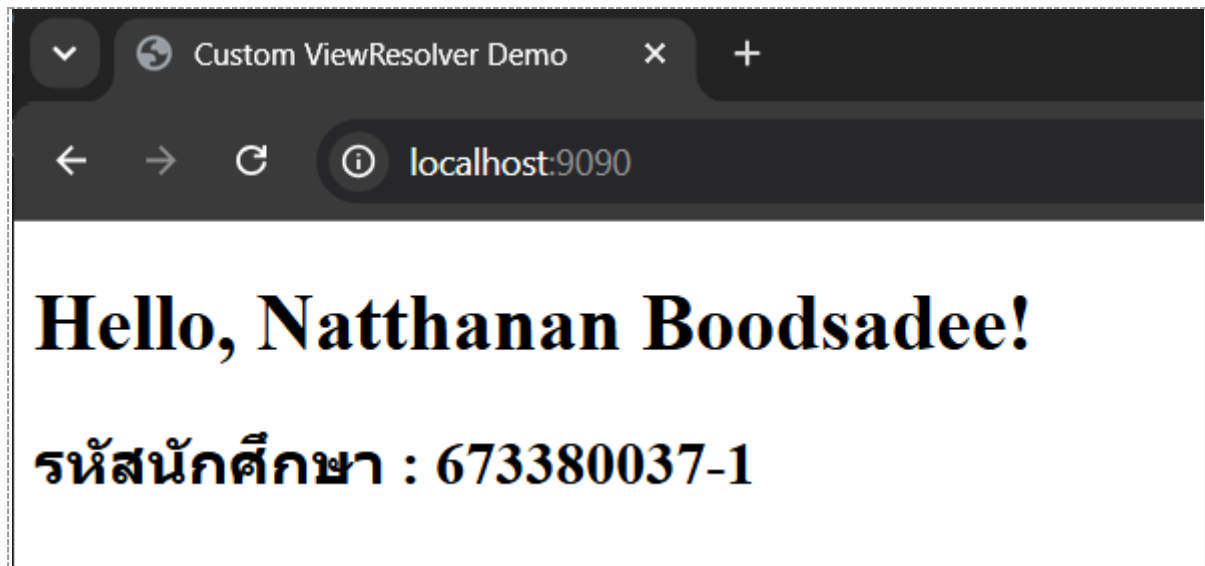
2. เพิ่มตัวแปรใหม่ชื่อ `studentId` ส่งจาก Controller ไปแสดงต่อจากชื่อในหน้า `home.html` (เช่น "สวัสดี ชื่อ (รหัส XXX)") แล้ว capture หน้าเว็บที่แสดงผลลัพธ์ (3 คะแนน)



3. เปลี่ยนค่า resolver.setPrefix() ใน ThymeleafConfig.java ให้ชี้ไปโฟลเดอร์ใหม่ชื่อ my-templates/ แทน custom-templates/ (ต้องย้ายไฟล์ home.html ไปด้วย) แล้ว capture ทั้งโค้ดที่แก้ และหน้าเว็บที่ยังทำงานปกติ (4 คะแนน)



4. เปลี่ยน server.port ใน application.properties เป็น 9090 แล้ว capture หน้าเว็บที่ URL เปลี่ยนเป็น http://localhost:9090/ (3 คะแนน)



5. เพิ่ม view ใหม่ชื่อ "about" (เพิ่ม @GetMapping("/about") และไฟล์ about.html)
ให้แสดงข้อความแนะนำตัวสั้นๆ แล้ว capture หน้าเว็บ /about ที่แสดงผลลัพธ์ (4 คะแนน)

